PROJECT REPORT · AI STUDENT ASSISTANT

CampusAI

A chat assistant for LPU students that answers questions about **your own data** (attendance, timetable, notices, messages, hostel leave) read live from the UMS portal, and **general university information** (rules, contacts, fees, exam dates) — in plain language, in 13 languages, by text or voice.

1 Problem and solution

The problem

- The UMS portal is slow and needs many clicks to find simple facts like attendance or the next class.
- Students must calculate by hand how many classes they can miss and still stay above 75%.
- General information (hostel fees, helpline numbers, exam dates, leave rules) is spread across many pages.

The solution

- One chat box: ask “Can I skip tomorrow?” or “What is my next class?” and get a direct answer.
- The student signs in to UMS once; CampusAI reads the data in the background so answers are instant.
- An AI model picks the right “tool”, gets the exact figures, and explains them clearly with charts if useful.

2 Key features

Live attendance

Overall and per-subject %, class-by-class log, and “how many can I miss / must I attend” for any target.

Timetable

Today’s classes, a given day, the class at a given time, and the next class with room number.

Notices & messages

Latest announcements and the full UMS message history, summarised on request.

Hostel leave

Reads leave balance and drafts a leave application. It never submits — the student stays in control.

University info

18 curated sections (rules, calendar, fees, contacts…) found by meaning with hybrid RAG search.

Voice & 13 languages

Hands-free voice mode (speak, listen to the answer) and replies in Hindi, Punjabi, Tamil, English and more.

Charts & diagrams

Answers can include bar charts, step timelines and flow diagrams rendered right inside the chat.

Chat memory

Conversations are saved; follow-ups like “and tomorrow?” work without repeating context.

Choice of AI model

Switch between Ollama Cloud, Gemini and OpenAI, with automatic fallback if one is down.

3 Project at a glance

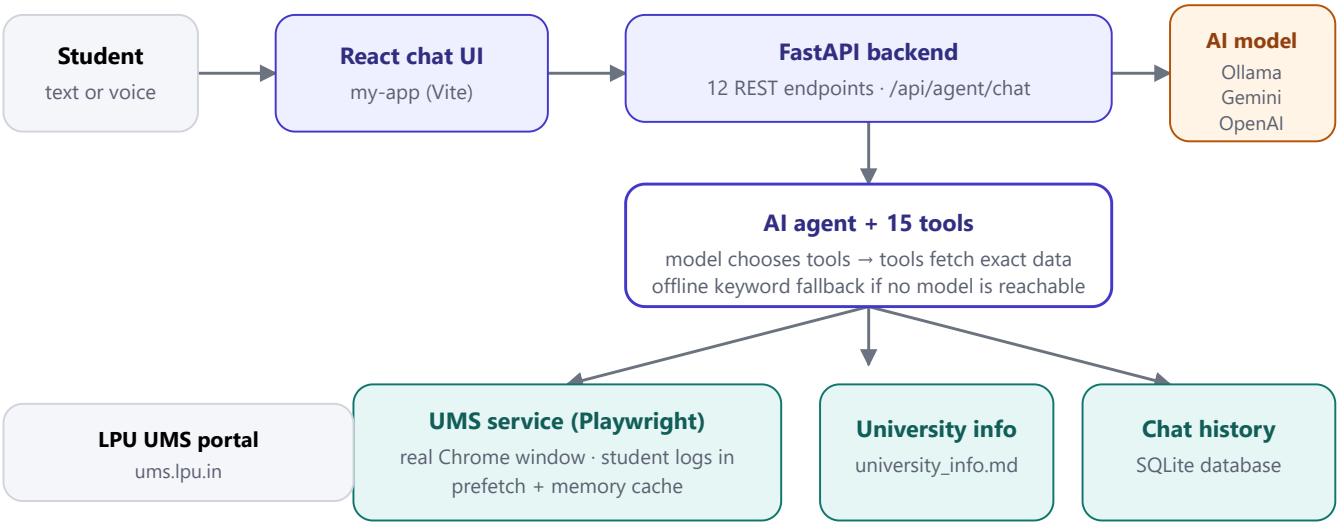
~11,100 lines of code	**15** AI tools	**3 + 1** AI providers + offline fallback	**13** reply languages
8,250 lines of Python (backend)	**2,900** lines of TypeScript (frontend)	**12** API endpoints	**18** university info topics

4 Technology stack

LAYER	TECHNOLOGIES
Frontend (chat UI)	React 19, TypeScript, Vite, Tailwind CSS, Framer Motion (animations), Recharts (charts), Mermaid (diagrams), Web Speech API (voice)
Backend (API server)	Python, FastAPI, Uvicorn, Pydantic (data validation), SQLAlchemy
AI models	Ollama Cloud (`gpt-oss:120b` , default), Google Gemini 2.5 Flash, OpenAI — all using "tool calling"
Portal automation	Playwright driving a real Chrome/Edge window over the Chrome DevTools Protocol
Knowledge search	Hybrid RAG: fastembed with the free local model `BAAI/bge-small-en-v1.5` (meaning) + keyword matching — no API key, works offline
Storage	SQLite file for chat history (no database server needed); a Markdown file for university facts

5 System architecture

The project has two parts: `my-app/` (the chat website) and `backend/` (the server with the AI agent). The diagram shows how they connect.



6 The AI tools

The model never guesses numbers. It can only answer from data returned by these tools, which all three AI providers share.

AREA	TOOLS	EXAMPLE QUESTION
Attendance	`get_attendance` , `get_attendance_detail` , `calculate_attendance`	"Am I safe in CSE329?"
Timetable	`get_timetable` , `get_day_classes` , `get_class_at` , `get_next_class`	"Where is my next class?"
Notices	`get_announcements` , `get_recent_announcements` , `get_messages`	"Any new notices this week?"
Hostel leave	`get_hostel_leave_balance` , `draft_hostel_leave`	"Draft a leave for Diwali."
Profile & info	`get_student_profile` , `get_university_info`	"What are the hostel fees?"
Diagnostics	`get_ums_diagnostics`	"Why can't you see my data?"

7 How one question is answered

1. Ask
"How many classes can I miss?" (typed or spoken)

2. Context
Backend adds recent chat turns + a summary of older ones.

3. Plan
The AI model picks tools: attendance, then the calculator.

4. Fetch
Tools return exact numbers from the UMS cache or live UMS.

5. Reply
Model writes the answer with its source; UI adds a chart.

Up to 5 tool rounds per question; every answer names its model and data source.

8 Attendance calculation

The "can I skip?" answer is pure maths done in Python, not by the AI, so it is always exact.

```
# Above target: largest x where attended / (total + x) ≥ target
can_miss   = floor(attended × 100 / target − total)
# Below target: smallest x where (attended + x) / (total + x) ≥ target
must_attend = ceil((target × total − 100 × attended) / (100 − target))
```

Example at 75%: with 45 of 56 classes attended (80.4%) a student can miss **4** more; with 38 of 56 (67.9%) they must attend the next **16** in a row. Duty leave is counted as present, matching university rules.

9 Privacy and safety by design

- **No passwords stored.** The student signs in to UMS by hand; CampusAI never reads or types credentials.
- **No bypassing security.** The student completes human verification; the app just waits.
- **Separate browser profile.** It never touches the student's normal Chrome profile.
- **Read-only.** Hostel leave is drafted, never submitted.
- **UMS data stays in memory**, cleared on disconnect, never saved to disk.
- **No stale figures.** Current numbers are always fetched fresh.

10 Challenges solved

CHALLENGE	HOW IT WAS SOLVED
Questions in different words	Hybrid RAG with a free local embedding model: "dorm rent" finds *Hostel fees*.
Slow answers from UMS	All data is **prefetched** right after sign-in and kept in a memory cache (refreshed every 15 min), so most answers are instant.
Hidden UMS pages	Attendance and timetable load only inside JavaScript pop-ups; the service clicks the right tile, reads the table, then closes it.
Long chats getting slow	Last 12 messages are sent word-for-word; older ones are compressed once into a summary, so cost stops growing.
AI provider outages	Automatic fallback to another model, then to a keyword-based agent that uses the same tools without any AI.

11 Future scope

- Deploy online so students can use it without running it locally.
- Mobile app and push reminders (e.g. "attendance below 75% in INT373").
- Add exam results, fee status and assignment deadlines from UMS.
- Automated tests for every tool and a fully free / local AI option.

Conclusion. CampusAI turns a slow, click-heavy portal into a simple conversation. It combines an AI model for understanding questions with exact, tool-based data for answers, while keeping the student's login and data private.